费曼口述：AI大模型、Spring AI——为什么要约束模型输出格式

说话人1: 费曼口述，大模型输出格式约束的底层逻辑、实现机制与Spring AI工程落地，共一百道关键问题，分十个阶段，从基础概念、为什么约束、结构化数据、约束机制、Spring AI落地、工程实现、异常可靠性、安全可信度、测试观测到架构面试，逐层讲透原理、边界与生产级实践。

一、基础概念与核心问题

说话人1: 问题一，什么是大模型的输出格式约束？为什么需要限制模型回答的格式，而不是让模型自由生成自然语言？
说话人1: 答，输出格式约束，就是给模型定输出契约，告诉它回答必须按指定的结构、字段、类型来，不能想到哪说到哪。
说话人1: 为什么要约束？因为输出给人看，自然语言没问题；但要给程序自动处理，自由文本就没法用了。程序不能像人一样听懂“大概意思”，它必须拿到确定的字段、固定的类型，才能往下走业务逻辑。你总不能让后端代码去模型的自然语言里抠“故障原因是啥”吧，那太脆弱了。

说话人1: 问题二，大模型生成的内容为什么可能出现格式不统一、字段缺失、类型错误、额外解释、内容截断或不符合业务要求的情况？
说话人1: 答，三个核心原因，本质是模型优先保证通顺，不保证严格符合格式。
说话人1: 第一，模型本质是概率生成，优先保证语言通顺，你没说死格式，它怎么顺手怎么来；第二，不同模型、不同版本对格式指令的服从度不一样，有的听话有的容易跑偏；第三，复杂任务下，模型容易加铺垫、加解释，甚至生成到一半就截断了。
说话人1: 说白了，模型的本职是生成语言，不是生成规范的数据结构，你不盯紧，它就自由发挥了。

说话人1: 问题三，大模型输出的自然语言、结构化文本、JSON、XML、Markdown 和 Java 对象分别是什么？它们处于什么不同的层次？
说话人1: 答，从自由到严格，是分层级的，越往后机器越容易处理，灵活性越差。
说话人1: 自然语言最自由，就是人话，灵活但机器难解析；结构化文本有固定框架，但还是文本；JSON、XML是标准结构化数据格式，字段清晰，机器能直接解析；Java对象是后端程序里的最终形态，数据映射成类实例，直接参与业务逻辑。
说话人1: 简单记：越往后面，机器越好处理；越往前面，人越容易读。

说话人1: 问题四，什么是结构化输出？它与普通文本生成有什么区别？
说话人1: 答，结构化输出，就是让模型输出严格符合预先定义的数据结构，固定字段、固定类型、固定格式，不是随便写一段话。
说话人1: 和普通文本生成的区别：普通文本目标是“通顺、合理”；结构化输出目标是“符合结构、字段完整、类型正确”。前者追求表达自然，后者追求格式合规。
说话人1: 一个是给人看的，一个是给程序用的，目标完全不同。

说话人1: 问题五，为什么自然语言对人类友好，却不一定适合程序直接处理？请结合 Java 后端接收模型返回值的场景进行分析。
说话人1: 答，人能理解歧义、能抓重点、能从一段话里提取自己要的信息；程序不行。
说话人1: 比如Java后端要拿故障诊断结果，要故障代码、风险等级、处理步骤这几个字段。模型回一句“建议你检查传感器，大概率是接触不良”，人能看懂，程序怎么拿？总不能每次都写正则表达式去抠关键词吧？
说话人1: 第一，正则写起来麻烦，维护成本高；第二，模型换个说法，正则就失效；第三，很容易提取错误，出了问题排查都没法排查。
说话人1: 所以后端程序处理，必须是结构化的，字段名固定、类型固定，才能直接反序列化成对象，走后续逻辑。

说话人1: 问题六，如果大模型回答的是“建议更换传感器”，而业务代码需要获得一个包含故障原因、处理步骤、风险等级的对象，会遇到什么问题？
说话人1: 答，会遇到“信息不可用、流程走不动”的问题，答案对是对，但程序用不了。
说话人1: 第一，信息粒度不够，“更换传感器”是结论，没有原因、没有步骤、没有风险等级，业务逻辑没法判断要不要派工单、走审批。
说话人1: 第二，没有结构化字段，程序不知道哪句是原因哪句是建议，总不能把整句话塞到“处理建议”字段里，那样数据质量就崩了。
说话人1: 第三，后续自动化流程走不了，比如自动创工单、自动派单，都需要结构化字段，自由文本根本喂不进去。

说话人1: 问题七，输出格式约束究竟是在约束模型的表达形式、数据结构、字段类型，还是业务语义？这些约束之间有什么区别？
说话人1: 答，约束的是表达形式、数据结构、字段类型，**不约束业务语义本身**。
说话人1: 区别：形式约束，管你怎么输出，比如必须JSON、必须有哪些字段、字段是什么类型；业务语义，管你说的内容对不对、准不准。
说话人1: 格式约束保证“长什么样”，不保证“说的对”。很多人搞混了，以为格式对了内容就对，完全两码事。

说话人1: 问题八，输出格式不符合要求，是否一定意味着模型给出的业务结论是错误的？反过来，格式完全正确是否意味着业务结论一定正确？
说话人1: 答，都不一定，两者是两个维度的问题。
说话人1: 格式不对，结论可能是对的，只是没按要求的格式说；比如模型说的很对，就是没套JSON，那是格式错，内容没错。
说话人1: 反过来，格式完全正确，内容也可能错，比如字段都齐了，但故障原因说错了，那是内容错，格式没错。
说话人1: 格式是载体，内容是核心。格式和内容是两个独立的评价维度。

说话人1: 问题九，为什么生产级 AI 应用不能将“模型能够返回答案”等同于“模型返回的答案能够直接用于业务处理”？
说话人1: 答，因为“能回答”是自然语言层面的，“能用于业务处理”是结构化、准确、合规层面的，差远了。
说话人1: 生产级业务处理，要求数据是确定的、字段是固定的、校验是可执行的、异常是可捕获的。模型自由回答，首先结构不确定，其次内容没校验，还有可能有幻觉，直接拿来用，哪天出问题都不知道。
说话人1: 就像你不能拿客服的聊天记录直接当财务凭证一样，格式、规范、校验都没有，不能进业务系统。

说话人1: 问题十，为什么输出格式约束既是模型交互问题，也是软件工程中的接口契约问题？
说话人1: 答，因为它本质就是应用和模型之间的接口约定。
说话人1: 模型交互层面：你告诉模型输出格式，模型按约定输出，两边才能对接上，不然各说各话。
说话人1: 软件工程层面：上下游系统之间，必须有明确的接口契约，输入输出是什么格式、什么字段、什么类型，大家都按契约来，才能集成。不然下游系统每次都要猜，那系统就太脆弱了。
说话人1: 说白了，模型也只是一个外部服务而已，当然也要遵守接口契约。

二、为什么需要约束模型输出格式

说话人1: 问题十一，从系统集成的角度看，输出格式约束解决了哪些问题：接口稳定性、数据解析、自动化执行、下游系统集成以及业务流程控制？
说话人1: 答，五个核心问题都解决了。
说话人1: 第一，接口稳定：格式约定好，接口就稳定了，不会模型一换说法，下游就崩了。
说话人1: 第二，数据解析：有固定格式，解析逻辑统一写一次就行，不用各种情况都适配。
说话人1: 第三，自动化执行：结构确定了，才能自动走后续业务流程，不用人工介入。
说话人1: 第四，下游集成：标准化的格式，才能方便对接其他系统，比如工单系统、设备管理系统。

说话人1: 第五，流程控制：字段固定了，才能做校验、做分支、做流程控制，不然数据都不确定，流程没法走。

说话人1: 问题十二，如果模型每次分别使用自然语言、Markdown 表格、编号列表和 JSON 回答同一个问题，下游 Java 程序可能遇到哪些困难？
说话人1: 答，那简直是灾难，三个层面的问题。
说话人1: 第一，解析逻辑写不下去，你总不能写N种解析逻辑吧，自然语言一种、Markdown一种、JSON一种，维护成本爆炸。
说话人1: 第二，异常满天飞，稍微换个格式，解析就报错，系统稳定性极差。
说话人1: 第三，业务逻辑没法做，字段时有时无，类型一会字符串一会数字，后续逻辑根本没法写，写了也到处是空指针、类型转换异常。
说话人1: 说白了，没有格式约束，下游程序就像在开盲盒，每次都不知道返回什么，根本没法工业化落地。

说话人1: 问题十三，在什么情况下，允许模型自由输出文本更合理？在什么情况下，必须采用严格的结构化输出？
说话人1: 答，看输出给谁用、要不要后续自动化处理。
说话人1: 如果只是给人看，比如聊天机器人回答用户问题、做科普、做咨询，自由文本就行，自然流畅，体验好。
说话人1: 如果是给程序用，要后续自动处理，比如自动创工单、自动分类、自动审批、写入数据库，必须严格结构化，不然程序接不住。
说话人1: 简单记：人看用文本，机用要结构化。

说话人1: 问题十四，为什么聊天机器人可以直接展示自然语言回答，而订单处理、工单创建、故障分类和自动审批等业务通常更需要结构化结果？
说话人1: 答：因为使用对象和后续流程不一样。
说话人1: 聊天机器人的终点是人，人能理解自然语言，灵活，有容错，只要意思对就行。
说话人1: 订单、工单这些业务场景，终点是系统、是流程、是数据库，必须要有确定的字段、固定的格式，才能自动流转、自动处理、自动存档。
说话人1: 越靠近自动化业务流程，对结构化要求越高；越靠近终端用户，越可以自由。

说话人1: 问题十五，如果希望模型生成一个设备故障诊断结果，为什么需要明确规定输出字段、允许值、字段类型和必要字段？
说话人1: 答，因为不规定死，模型输出就会“自由发挥”，程序没法用。
说话人1: 不规定字段，模型可能漏了故障代码，或者多了一堆没用的解释；
说话人1: 不规定允许值，故障等级可能给你整个“中等偏高”这种描述，程序没法枚举匹配；
说话人1: 不规定字段类型，置信度可能给你写个“百分之八十”，不是数字，没法计算；
说话人1: 不规定必要字段，关键的处理建议可能直接没了。
说话人1: 只有都规定清楚，输出才是可控的、可预期的，程序才能稳定消费。

说话人1: 问题十六，为什么明确要求“输出 JSON”仍然不一定足够？模型可能返回哪些看似是 JSON、实际却无法被标准解析器接受的内容？
说话人1: 答，因为“JSON”的范围太宽了，模型理解的JSON和程序要的JSON可能不是一回事。
说话人1: 常见的坑：第一，前后加解释文字，比如“好的，我给你返回JSON：\n{...}\n希望对你有帮助”，外面包了人话，直接解析肯定失败。
说话人1: 第二，用Markdown代码块包JSON，就是`json ...`，很多模型喜欢这么干，标准解析器不认。
说话人1: 第三，语法错误，字段不闭合、引号转义错、多了逗号少了逗号，解析失败。
说话人1: 第四，类型不对，数字加引号、布尔值写字符串，看似JSON，实际类型全错。
说话人1: 所以只说输出JSON，远远不够。

说话人1: 问题十七，为什么在 Prompt 中写明“只能输出 JSON，不要输出其他内容”，不能被视为绝对可靠的格式保证？
说话人1: 答，因为模型的服从度不是100%，尤其是小模型，或者上下文长的时候，很容易跑偏。
说话人1: 第一，模型可能忘了，或者执行不到位，说着说着就加了额外解释。

说话人1: 第二，不同模型对指令的遵循度不一样，有的很听话，有的就喜欢加前缀后缀。
说话人1: 第三，复杂任务下，模型可能先思考、先解释，再输出JSON，控制不住。
说话人1: 所以这只是个软约束，不是硬保证，不能100%依赖，必须有后端校验兜底。

说话人1: 问题十八，当模型返回了 Markdown 代码围栏包裹的 JSON 时，为什么某些解析流程能够接受，而另一些流程会失败？
说话人1: 答，因为解析逻辑不一样，有没有做前置清洗的区别。
说话人1: 如果解析前先做了清洗，把`json、`这些标记去掉，只提取中间的JSON部分，那就能解析成功；
说话人1: 如果直接拿完整字符串喂给JSON解析器，那肯定失败，因为代码围栏标记不是JSON语法的一部分。
说话人1: 这就是为什么很多时候你看着像JSON，就是解析失败，因为外面包了“壳”。生产级的解析一般都要有清洗步骤，但也要注意不能乱清洗，别把内容改了。

说话人1: 问题十九，为什么格式统一能够降低系统的解析复杂度、分支判断数量和维护成本？这种收益在哪些复杂业务场景中最明显？
说话人1: 答，因为格式统一了，解析逻辑就一套，不用处理各种情况，分支少了，bug就少了，维护成本自然降了。
说话人1: 你想想，如果模型可能返回文本、列表、表格、JSON四种格式，你就要写四套解析逻辑，还要写判断逻辑判断是哪种，还要处理各种边界情况，维护量翻好几倍。格式统一了，就一套解析逻辑，出问题也好排查。
说话人1: 越复杂的业务场景、越多的模型供应商、越频繁的需求变更，这个收益就越明显。格式统一，是降低系统复杂度的核心手段之一。

说话人1: 问题二十，如果没有输出格式约束，开发者通过正则表达式、字符串截取或关键词判断来提取答案，会引入哪些稳定性和可维护性问题？
说话人1: 答，三个大坑，越往后越严重。
说话人1: 第一，稳定性差，模型稍微改个说法、换个表达方式，正则就失效了，错误率飙升。
说话人1: 第二，维护成本高，每次模型升级、换模型，都要重写提取逻辑，跟着改，工作量巨大。
说话人1: 第三，可解释性差，出了问题，你都不知道是正则写错了，还是模型输出变了，排查困难。
说话人1: 说白了，这就是在脆弱的协议上堆脆弱的逻辑，系统一碰就碎，生产级系统绝对不能这么干。

三、从自由文本到结构化数据

说话人1: 问题二十一，什么是自由文本输出？它适合哪些业务任务，又有哪些天然局限？
说话人1: 答，自由文本输出，就是模型不受结构限制，自然生成的文本内容，和正常说话一样。
说话人1: 适合的任务：面向用户的对话、科普解释、创意生成、头脑风暴，这些给人看的、不需要后续自动化处理的场景。
说话人1: 天然局限：结构不确定、字段不固定、没法直接被程序解析，只能人工阅读，不适合自动化流程。
说话人1: 简单说：适合人看，不适合机用。

说话人1: 问题二十二，什么是固定格式输出？如果模型必须按照指定模板生成回答，模板应当明确到什么程度？
说话人1: 答，固定格式输出，就是要求模型必须按照指定的模板、结构、字段来生成回答，不能自由发挥。
说话人1: 模板要明确到什么程度？四个层面：第一，整体结构，比如分几部分；第二，字段名称，必须有哪些字段，叫什么；第三，字段类型，是字符串、数字、布尔还是数组；第四，约束规则，比如枚举值、取值范围、是否必填。
说话人1: 越明确，输出越可控；越模糊，发挥空间越大，出错概率越高。

说话人1: 问题二十三，什么是 JSON 输出？为什么 JSON 特别适合在 Java 后端与大模型之间传递结构化数据？
说话人1: 答，JSON输出，就是让模型用JSON格式输出结构化数据，键值对形式，字段清晰。
说话人1: 为什么适合Java后端？第一，JSON是前后端、服务间交互的标准格式，Java生态有成熟的Jackson工具，解析方便；第二，结构化，字段名、类型都清晰，能直接映射成Java对象，不用自己抠；第三，跨语言、跨厂商通用，不管什么模型，只要输出JSON，Java都能接。
说话人1: 相当于在模型和Java程序之间，搭了一个标准的数据桥梁。

说话人1: 问题二十四，JSON 格式约束与 JSON Schema 约束有什么区别？仅要求返回 JSON，与要求返回符合特定 Schema 的 JSON，差异在哪里？
说话人1: 答，差远了，前者只要求语法合法，后者要求内容结构都符合约定。
说话人1: JSON格式约束，只要求输出是合法的JSON语法，括号引号对对就行，里面有什么字段、字段什么类型不管。
说话人1: JSON Schema约束，不仅要是合法JSON，还必须符合Schema定义：有哪些字段、字段是什么类型、哪些必填、枚举值有哪些、嵌套结构什么样。
说话人1: 一个是语法层面的及格线，一个是契约层面的优秀线，严格程度完全不一样。

说话人1: 问题二十五，什么是 JSON Schema？它如何描述对象属性、字段类型、必填字段、枚举值、数组结构和嵌套对象？
说话人1: 答，JSON Schema就是JSON的结构定义契约，用来描述一个JSON必须长什么样。
说话人1: 怎么描述：用properties定义属性字段，type指定类型，比如string、number、boolean、array、object；required数组指定哪些字段必填；enum指定枚举允许值；items定义数组元素的类型；properties定义嵌套对象的内部字段。
说话人1: 相当于给JSON输出画了个模子，模型输出必须套进去，差一点都不行。有了Schema，就能做自动校验，输出合不合格，一验就知道。

说话人1: 问题二十六，如果业务要求模型输出以下信息：设备编号、故障代码、故障原因、置信度和处理建议，应如何把自然语言需求转换成可验证的数据结构？
说话人1: 答，四步走，把模糊需求变成可验证的结构。
说话人1: 第一步，梳理字段：把需求里的信息点都列出来，每个字段明确业务含义。
说话人1: 第二步，定类型：设备编号、故障代码是字符串；置信度是数字，0到1之间；故障原因、处理建议是字符串。
说话人1: 第三步，定规则：哪些是必填的，比如设备编号、故障代码必须有；哪些有枚举限制，比如故障等级只能是低中高。
说话人1: 第四步，写成JSON Schema，或者直接定义成Java Bean，用结构定义约束输出。
说话人1: 最终目标：输出的结果可以直接校验，合不合格一目了然。

说话人1: 问题二十七，在选择输出结构时，为什么需要明确区分字符串、数字、布尔值、数组、对象和可空字段？
说话人1: 答，因为类型不对，Java程序直接就报错了，业务逻辑也没法走。
说话人1: 比如故障等级，你以为是数字，模型给你返回字符串“高级”，类型转换直接失败；
说话人1: 比如是否需要派工，你以为是布尔值，模型给你写个“需要”，没法直接判断；
说话人1: 比如多个故障原因，你以为是数组，模型给你一段逗号分隔的字符串，没法遍历。
说话人1: 可空字段更是，如果约定不明确，模型有时候省略有时候给null，程序很容易空指针。
说话人1: 类型是数据的基础，基础不牢，地动山摇。

说话人1: 问题二十八，如果一个字段允许为空，应该使用 null、空字符串、空数组，还是直接省略字段？如何根据业务语义确定规则？
说话人1: 答，没有统一答案，看业务语义，必须约定清楚，不然就乱。
说话人1: null：代表“不存在、没有值”，比如没有故障原因，就用null，语义清晰。
说话人1: 空字符串：代表值存在但是为空，比如备注为空，适合文本字段。
说话人1: 空数组：代表集合为空，比如没有历史故障，适合数组字段。
说话人1: 省略字段：一般不推荐，因为程序里字段缺失和值为空是两回事，容易引发反序列化问题，也不好校验。

说话人1: 规则就是：语义对应类型，数组空就用空数组，没有值就用null，尽量不省略字段，保证结构稳定。

说话人1: 问题二十九，如果模型需要返回多个故障原因，每个原因都包含描述、证据与风险等级，应该如何设计嵌套结构，避免生成难以解析的数据?
说话人1: 答，用数组加对象的嵌套结构，统一定义内部对象结构，清晰规范。
说话人1: 首先，定义一个内部对象，比如FaultReason，包含description描述、evidence证据、riskLevel风险等级三个字段，每个字段定好类型和规则。
说话人1: 然后，外层用一个数组，类型是FaultReason数组，比如reasons: FaultReason[]。
说话人1: 同时在Prompt里明确说明：多个原因用数组返回，每个原因包含三个字段，不要用自然语言罗列。
说话人1: 这样设计，程序直接解析成对象数组，遍历处理就行，结构清晰，不容易乱。

说话人1: 问题三十，当模型输出包含未知字段、缺失字段、重复字段或字段类型不匹配时，Java 应用应该如何处理?
说话人1: 答，分情况处理，分级兜底，不能一错就崩，也不能稀里糊涂就过。
说话人1: 第一，未知字段：可以配置Jackson忽略未知字段，不报错，兼容扩展，但要日志告警。
说话人1: 第二，缺失必填字段：属于校验失败，抛异常，触发重试或者降级。
说话人1: 第三，重复字段：按JSON标准处理，一般后面覆盖前面，同时记录告警。
说话人1: 第四，类型不匹配：尝试温和转换，转不了就抛校验异常。
说话人1: 核心原则：容错但可控，异常有记录，不能静默失败，也不能随便崩溃。

四、输出约束的不同机制

说话人1: 问题三十一，通过 Prompt 指令约束输出格式，与通过模型 API 的结构化输出参数约束格式，有什么区别?
说话人1: 答，一个是软约束，靠模型自觉；一个是硬约束，靠API机制强制。
说话人1: Prompt约束：就是在提示词里告诉模型要什么格式，全靠模型自己理解执行，服从度看模型能力，没有强制力，容易跑偏。
说话人1: API结构化参数约束：是模型服务端原生支持的功能，比如JSON Mode，开启之后服务端会强制输出合法JSON，不是模型自己想不想的问题，硬约束，可靠性高很多。
说话人1: 一个是“请你按格式输出”，一个是“我开启强制格式模式”，力度完全不一样。

说话人1: 问题三十二，什么是 JSON Mode？它主要解决什么问题，又有哪些问题仍然需要应用自行检查?
说话人1: 答，JSON Mode是很多大模型服务商提供的原生功能，开启之后，模型服务端会强制保证输出的是合法的JSON格式，不会有多余的自然语言。
说话人1: 主要解决的问题：就是模型输出包着自然语言、格式不对、解析失败的问题，大幅提升合法JSON的概率。
说话人1: 但它不是万能的：它只保证语法合法，不保证字段齐全、不保证类型正确、不保证内容符合业务要求。这些还是要应用自己校验。
说话人1: 说白了，它管“是不是JSON”，不管“JSON对不对”。

说话人1: 问题三十三，什么是基于 Schema 的结构化输出？它与仅要求模型返回合法 JSON 的机制有什么区别?
说话人1: 答，基于Schema的结构化输出，就是把JSON Schema传给模型，模型服务端强制输出符合这个Schema的JSON，不仅语法对，结构、字段、类型都要对。
说话人1: 和只要求合法JSON的区别：前者是结构契约级约束，后者只是语法级约束。
说话人1: 只要求JSON，只要括号对就行，里面字段随便来；Schema约束，字段多了少了、类型错了，都不行，必须严格符合定义。
说话人1: 一个是及格线，一个是优秀线，严格程度完全不一样。

说话人1: 问题三十四，什么是受约束解码？它与单纯在 Prompt 中描述输出格式的实现路径有什么区别？
说话人1: 答，受约束解码也叫Constrained Decoding，是在解码阶段就限制模型输出的token，只能生成符合指定语法、结构的内容，不符合的token直接不让生成。
说话人1: 和Prompt描述的区别：Prompt是让模型自己理解着来，属于“软提示”；受约束解码是在生成底层就卡死，不符合规则的token直接不生成，属于“硬限制”，可靠性高得多。
说话人1: 一个是思想引导，一个是物理限制，强制力不在一个级别。

说话人1: 问题三十五，输出格式约束是否总能保证模型返回完全符合业务要求的数据？哪些问题仍然需要由应用程序检查？
说话人1: 答，当然不能100%保证，格式约束只是大幅提升合规率，不是万能的。
说话人1: 应用还要自己查的：第一，业务语义校验，比如故障代码是不是存在、设备编号对不对，格式约束管不了内容对错；
说话人1: 第二，字段完整性校验，必填字段有没有，即使Schema约束，也要自己再验一遍保险；
说话人1: 第三，枚举值范围校验，是不是在业务允许的取值里；
说话人1: 第四，异常兜底，万一格式还是错了，要有降级、重试机制。
说话人1: 格式约束是帮手，不是甩锅的借口，最终责任还是在应用。

说话人1: 问题三十六，不同的大模型供应商对 JSON 模式、JSON Schema、结构化响应和受约束解码的支持可能有什么差异？为什么不能假设所有模型具有相同的能力？
说话人1: 答，差异非常大，支持程度参差不齐。
说话人1: 有的支持JSON Mode，有的不支持；有的支持Schema约束，有的只支持JSON格式；有的支持受约束解码，有的只能靠Prompt。
说话人1: 甚至同一家的不同模型版本，支持程度都不一样。
说话人1: 所以不能假设所有模型都有同样的能力，要做兼容层，不能把业务逻辑绑死在某个供应商的专有功能上。不然换个模型，系统直接崩了。

说话人1: 问题三十七，如果当前模型不支持严格的 Schema 约束，应该如何设计兼容方案？Prompt 约束、解析、校验和有限重试各自负责什么？
说话人1: 答，做四层兼容方案，层层兜底，不依赖单一机制。
说话人1: 第一层，Prompt约束：明确要求输出格式、字段、类型，尽量引导模型输出正确结构。
说话人1: 第二层，解析修复：解析前做清洗，去Markdown标记、去多余文本，尝试修复小问题。
说话人1: 第三层，格式校验：解析之后做Schema校验，字段、类型、必填，合不合格一目了然。
说话人1: 第四层，有限重试：校验失败的话，把错误原因告诉模型，让它重生成，最多重试2-3次。再不行就抛异常，走降级逻辑。
说话人1: 这样即使没有原生Schema支持，也能达到很高的成功率。

说话人1: 问题三十八，输出格式约束和 Tool Calling（工具调用）有什么区别？工具调用中的参数结构与普通文本响应中的 JSON 结构有何不同？
说话人1: 答，完全是两个东西，目的、流程、场景都不一样。
说话人1: 输出格式约束：约束模型回答的格式，让输出结构化，方便程序处理结果，是返回结果的格式约定。
说话人1: Tool Calling：是让模型决定要不要调用工具、调用哪个工具、传什么参数，是模型的能力扩展。
说话人1: 不同点：工具调用的参数结构，是模型用来调用函数的，有固定的函数名、参数结构，是模型和工具之间的契约；普通JSON结构是业务返回结构，是应用和模型之间的结果契约。
说话人1: 一个是“你要调用我什么工具”，一个是“你要返回我什么结果”，两码事。

说话人1: 问题三十九，输出格式约束与 Function Calling 是否属于同一个概念？在实际开发中应该如何区分它们的目的与执行流程？
说话人1: 答，不是一个概念，完全两码事。
说话人1: 目的区别：格式约束目的是让模型输出结构化的回答，方便程序处理结果；函数调用目的是让模型调用外部工具，扩展能力。
说话人1: 流程区别：格式约束是输出的时候约束结果格式；函数调用是模型生成工具调用请求，应用执

行完工具再把结果返回模型。
说话人1: 实际开发中：结构化输出用在最终结果返回，给业务层用；工具调用用在中间步骤，让模型可以调用外部能力。不要混为一谈。

说话人1: 问题四十，为什么仅仅在模型返回的文本中看到了类似 JSON 的内容，并不能证明模型服务端已经执行了严格的结构化输出约束？
说话人1: 答：因为看起来像JSON，和服务端强制生成JSON，完全是两回事。
说话人1: 模型自己理解了Prompt，也能输出很标准的JSON，但这是它自己生成的，不是服务端强制的，下次可能就变了。
说话人1: 严格的结构化输出约束，是服务端在解码阶段就限制死了，不管模型怎么想，都只能输出合法格式，稳定性高得多。
说话人1: 你看到的JSON只是结果，过程有没有强制，从结果上不一定看得出来，不能凭外观判断机制。

五、Spring AI 中的结构化输出

说话人1: 问题四十一，Spring AI 为什么需要提供结构化输出相关能力？它解决了 Java 开发者直接处理大模型文本时的哪些问题？
说话人1: 答，因为Java是强类型语言，开发者最想要的就是拿到直接能用的Java对象，不是自己抠字符串。
说话人1: 它解决的问题：第一，不用自己写解析逻辑，不用处理各种格式异常，框架帮你做了；第二，不用自己和不同厂商的结构化功能对接，框架统一封装，换模型不用改业务代码；第三，统一的类型映射机制，直接转换成Java Bean，开发效率高；第四，统一的异常处理，解析失败、类型不匹配，都有统一的异常。
说话人1: 说白了，就是把结构化输出这件脏活累活，框架给你包了，开发者专心写业务就行。

说话人1: 问题四十二，在 Spring AI 的典型调用流程中，ChatClient、ChatModel、Prompt、Message 与结构化输出转换组件分别承担什么职责？
说话人1: 答，分层协作，各管一段。
说话人1: ChatClient：高层调用入口，开发者用的，提供流式、结构化这些高级API。
说话人1: ChatModel：底层模型抽象，对接厂商API，负责原生的请求响应。
说话人1: Prompt：单次请求的输入封装，包含所有消息、参数、格式要求。
说话人1: Message：单条消息，角色加内容。
说话人1: 结构化输出转换组件：负责把模型返回的文本，转换成指定的Java对象，做解析、校验、映射。
说话人1: 简单说：业务调用ChatClient，传Prompt，底层ChatModel调模型，返回结果后转换组件转成Java对象。

说话人1: 问题四十三，在 Spring AI 中，如何让模型的响应从普通文本转换成 Java 对象？这一过程涉及哪些步骤？
说话人1: 答，完整流程分五步。
说话人1: 第一步，开发者指定目标Java类型，调用entity()之类的方法；
说话人1: 第二步，Spring AI把目标类型转换成格式要求，加到Prompt里，告诉模型要输出什么格式的JSON；
说话人1: 第三步，调用大模型API，拿到返回的文本内容；
说话人1: 第四步，结构化输出转换器，把返回的JSON解析成目标Java类型的对象；
说话人1: 第五步，返回给业务代码，直接用对象就行。
说话人1: 整个过程对开发者透明，大部分时候只要指定类型就行，不用自己写解析。

说话人1: 问题四十四，Spring AI 中的 entity() 这类类型映射方式解决什么问题？它与直接调用 content() 获取文本有什么区别？
说话人1: 答，解决的就是“把模型输出转成Java对象”的问题，不用自己解析JSON。
说话人1: 区别：content()拿到的是原始字符串，你要自己解析、自己处理异常、自己转对象，什么都要自己来。

说话人1: entity()直接返回指定类型的Java对象，框架帮你做了格式要求、解析、类型转换，你直接拿对象用就行。
说话人1: 就像你去餐厅，content()是给你原材料自己做，entity()是直接给你做好的菜，直接吃。

说话人1: 问题四十五，Spring AI 的 StructuredOutputConverter 及相关实现类主要解决什么问题？它们与模型本身支持的原生结构化输出之间有什么区别？
说话人1: 答：核心解决模型输出到Java对象的转换问题，是框架的转换层。
说话人1: 和模型原生结构化输出的区别：模型原生结构化输出是服务端的能力，保证输出格式正确；StructuredOutputConverter是Spring AI这边的转换组件，负责把模型返回的内容，不管是原生结构化的还是纯文本的，转换成Java对象。
说话人1: 也就是说，模型支持原生结构化，转换器就省点事；模型不支持，转换器也会尽量解析、尽量转。
说话人1: 一个是模型端的能力，一个是框架端的适配，两者配合工作。

说话人1: 问题四十六，BeanOutputConverter 的职责是什么？它如何利用目标 Java 类型帮助确定结构化输出的目标形态？
说话人1: 答，BeanOutputConverter就是专门负责把模型输出转换成Java Bean对象的转换器。
说话人1: 怎么利用目标类型：第一，它会读取目标Java类的结构，字段名、类型、注解，生成对应的格式要求；第二，把格式要求注入到Prompt里，告诉模型按这个输出；第三，拿到模型返回后，用Jackson把JSON反序列化成目标Java对象；第四，做基本的类型校验，有问题抛异常。
说话人1: 相当于它拿着Java类当模板，去要求模型输出格式，再把结果套进类里。

说话人1: 问题四十七，当开发者通过 Java 类定义期望的输出结构时，Spring AI 如何将类型信息与模型请求、响应解析关联起来？
说话人1: 答，双向关联，请求端用类型生成格式约束，响应端用类型做映射解析。
说话人1: 请求侧：读取目标Java类的字段、类型、注解，生成对应的JSON结构要求，加到Prompt里，告诉模型按这个输出。
说话人1: 响应侧：拿到模型返回的文本，用目标类型作为目标，用Jackson做反序列化，转换成对应对象。
说话人1: 简单说：请求时，照着类的样子提要求；响应时，照着类的样子解析结果。类就是模板，两头对应。

说话人1: 问题四十八，Java Bean、Java Record 和其他 Java 类型作为结构化输出目标时，各自有哪些优缺点？应如何选择？
说话人1: 答，各有优劣，看场景。
说话人1: Java Bean：优点是灵活，可加方法、可加校验注解、可修改；缺点是代码多，要写getter setter，或者用Lombok。
说话人1: Java Record：优点是代码简洁，不可变，天生适合数据传输，默认自带构造方法、equals、hashCode；缺点是灵活性差，不能继承，字段不能改。
说话人1: 选型：纯数据传输、不需要修改，用Record，简洁干净；需要业务逻辑、需要修改、需要加自定义校验，用Bean。

说话人1: 问题四十九，如果返回对象包含嵌套对象、集合、枚举或复杂泛型，应该重点检查哪些序列化、反序列化和类型映射问题？
说话人1: 答，四个高风险点，很容易出问题。
说话人1: 第一，嵌套对象：要确保内层对象也有正确的结构定义，字段名类型匹配，不然内层解析失败。
说话人1: 第二，集合类型：比如List，要确保泛型类型能被正确识别，不然集合里的元素类型不对。
说话人1: 第三，枚举类型：要确保模型返回的字符串和枚举值能对应上，大小写、命名一致，不然转换失败，最好加自定义序列化器。
说话人1: 第四，复杂泛型：比如嵌套的泛型结构，要确认框架的类型推断能处理，不行就用具体类型，别搞太复杂。
说话人1: 越复杂的结构，越要做完整的测试，不能想当然认为能转。

说话人1: 问题五十，使用 Spring AI 的输出转换器，是否意味着模型一定会严格遵循指定格式？转换器能够保证哪些事情，又不能保证哪些事情？
说话人1: 答，当然不能保证模型一定遵循，转换器不是万能的。
说话人1: 能保证的：第一，会把格式要求发送给模型；第二，返回结果会尝试解析成目标类型；第三，解析失败会抛异常，不会静默返回错误对象。
说话人1: 不能保证的：第一，不能保证模型一定按格式输出，模型不听话它也没办法；第二，不能保证内容正确，只保证格式类型匹配；第三，不能保证业务逻辑正确，只管结构映射。
说话人1: 转换器是工具，不是魔法，最终输出怎么样，还是要看模型配不配合。

说话人1: 问题五十一，Spring AI 的输出转换器、模型 API 的结构化输出能力和 Jackson 等 JSON 处理组件之间是什么关系？它们是否可以互相替代？
说话人1: 答，是协作关系，各管一层，不能互相替代。
说话人1: 模型API结构化能力：是服务端的，保证输出格式合法，是最底层的保障。
说话人1: Jackson：是Java的JSON处理工具，负责序列化反序列化，是转换的基础工具。
说话人1: Spring AI输出转换器：在两者之上，封装了“要求模型输出格式 + 解析结果 + 类型映射”的完整流程，串起来用。
说话人1: 相当于：Jackson是砖头，模型结构化能力是地基，Spring AI转换器是盖好的房子。房子用砖头，但砖头不等于房子。

说话人1: 问题五十二，如何确认项目中使用的 Spring AI 版本究竟支持哪些结构化输出能力、配置项与 API？应该如何核实官方文档、依赖版本和实际运行行为？
说话人1: 答，三步确认，不能靠博客教程，要以官方为准。
说话人1: 第一步，查对应版本的官方参考文档，看结构化输出章节，有哪些类、哪些API、哪些配置，类名、包名对不对。
说话人1: 第二步，看依赖的版本，不同大版本变化很大，1.0和1.1可能不一样，确认版本号。
说话人1: 第三步，写单元测试验证，实际调用一下，看返回类型对不对、异常正不正常，跑一遍最准。
说话人1: 核心原则：官方文档为准，实际验证为依据，不要道听途说。

说话人1: 问题五十三，当 Spring AI 的结构化输出转换失败时，如何区分是 Prompt 设计问题、模型响应问题、解析器问题、Java 类型定义问题，还是供应商功能差异？
说话人1: 答，按从易到难、从外到内的顺序排查，逐个排除。
说话人1: 第一步，先看模型返回的原始文本是什么，格式对不对？如果原始文本格式就错，那就是Prompt或者模型的问题。
说话人1: 第二步，原始文本格式对，那看Java类型定义对不对，字段名、类型是不是不匹配，比如字段名拼错了。
说话人1: 第三步，类型也对，那看解析器配置，比如Jackson配置，是不是未知字段报错、枚举不匹配。
说话人1: 第四步，都没问题还失败，那就是供应商功能差异，换个模型试试，或者换原生结构化模式。
说话人1: 一步步缩小范围，总能定位根因。

六、Java 工程实现与代码阅读

说话人1: 问题五十四，假设需要开发一个设备故障分类接口，模型必须返回故障类别、故障等级、原因和处理建议，你会如何定义 Java 输出模型？
说话人1: 答，定义成Java Record或者Bean，字段明确，加校验注解。
说话人1: 比如用Record：

```
public record FaultDiagnosis(
  @NotBlank String faultCategory,
  @NotNull Integer faultLevel,
  @NotBlank String faultReason,
  @NotBlank String suggestion
) {}
```

说话人1: 或者用Lombok的@Data类，加上Bean Validation注解，比如@NotNull、@NotBlank，甚至@Min@Max。
说话人1: 字段名和业务含义对应，类型准确，必填明确，后面解析、校验都能用。

说话人1: 问题五十五，如果让模型返回 FaultDiagnosis 对象，如何从调用入口追踪到请求构造、模型响应、字符串提取、JSON 解析和对象映射的完整代码链路?
说话人1: 答，完整链路分七步。
说话人1: 第一步，业务Service注入ChatClient，准备用户问题。
说话人1: 第二步，构建Prompt，包含系统提示、用户问题，指定输出类型为FaultDiagnosis.class。
说话人1: 第三步，调用ChatClient的entity方法，传入Prompt和目标类型。
说话人1: 第四步，Spring AI内部，把目标类型转换成格式约束，加入请求，调用ChatModel。
说话人1: 第五步，模型返回结果，StructuredOutputConverter拿到响应文本。
说话人1: 第六步，转换器用Jackson把JSON解析成FaultDiagnosis对象，做基本类型校验。
说话人1: 第七步，返回FaultDiagnosis对象给业务代码，直接使用。
说话人1: 整个过程大部分透明，业务代码只要指定类型就行。

说话人1: 问题五十六，在 Spring AI 项目中，如果返回结果使用 String 与使用 .entity(FaultDiagnosis.class) 等类型映射方式，业务代码的控制流程和异常处理会有哪些差异?
说话人1: 答，差异非常大，一个是手动全控，一个是框架代劳。
说话人1: 用String：业务代码拿到字符串，自己写解析、自己处理异常、自己校验，灵活但工作量大，所有异常都要自己捕获处理。
说话人1: 用entity()：框架帮你解析、转换，成功直接拿对象，失败抛异常，业务代码只要处理正常业务逻辑和捕获转换异常就行，代码简洁很多。
说话人1: 但注意，框架只做格式类型校验，业务规则校验还是要自己做。
说话人1: 简单说：省事就用entity，要极致灵活就自己拿String处理。

说话人1: 问题五十七，如果模型返回了合法 JSON，但某个字段的类型与 Java 属性不匹配，会在处理流程的哪个阶段失败？应该如何定位?
说话人1: 答：在JSON反序列化阶段失败，也就是转换器做Jackson反序列化的时候报错。
说话人1: 怎么定位？第一，看异常堆栈，Jackson的异常会明确告诉你哪个字段、类型不匹配，比如Cannot deserialize value of type。
说话人1: 第二，开启模型原始响应日志，看返回的JSON实际是什么类型，比如数字写成了字符串。
说话人1: 第三，对照Java类定义，看字段类型是不是对应，比如int字段返回了带引号的数字。
说话人1: 定位非常快，Jackson的异常信息还是很明确的，只要你会看。

说话人1: 问题五十八，如果 Java 对象定义了必填字段，但模型返回时遗漏了这些字段，Java 对象反序列化成功是否就意味着业务校验成功？为什么需要区分解析和校验?
说话人1: 答：当然不是，反序列化成功只代表格式上能转成对象，不代表业务上合法。
说话人1: 因为JSON反序列化，默认情况下，缺失的字段就是null，只要类型对就不会报错，但业务上必填字段为空是不合法的。
说话人1: 所以要区分：解析，只看格式、类型对不对，能不能转成对象；校验，看业务规则合不合法，必填有没有、值在不在范围内。
说话人1: 解析是基础，校验是业务保障。解析过了不代表业务对，必须再加Bean Validation或者手动校验。

说话人1: 问题五十九，如何利用 Jackson 将模型输出转换为 Java 类型？在配置未知字段处理、枚举解析、日期格式和空值策略时需要注意哪些问题?
说话人1: 答，用ObjectMapper的readValue方法转，配置要注意四个点。
说话人1: 第一，未知字段：建议配置FAIL_ON_UNKNOWN_PROPERTIES关闭，避免模型多字段就报错，但要打日志告警。
说话人1: 第二，枚举：最好用@JsonValue注解自定义枚举序列化反序列化，或者配置READ_UNKNOWN_ENUM_VALUES_AS_NULL，避免未知枚举值直接崩。
说话人1: 第三，日期：统一日期格式，用@JsonFormat注解，避免不同模型日期格式不一样。
说话人1: 第四，空值策略：明确null和缺失字段的处理，比如用@JsonInclude控制序列化，反序列化默

认null。
说话人1: 配置合理，能大幅减少解析异常，同时保留可控性。

说话人1: 问题六十，如何使用 Bean Validation（Bean 校验）为模型输出定义 @NotNull、@NotBlank、@Min、@Max、枚举约束或自定义业务规则?
说话人1: 答，给Java输出类的字段上加对应注解，然后用Validator手动校验，或者在Controller层自动校验。
说话人1: 比如：字段上加@NotNull，不能为null；@NotBlank，字符串不能空；@Min(1)@Max(5)，数字在1到5之间；枚举的话，用@Pattern或者自定义校验注解。
说话人1: 然后业务代码拿到对象后，用Validator校验，返回校验结果，不合格就抛异常或者重试。
说话人1: 相当于给输出对象加了一道业务规则安检，格式对了还不够，规则也要符合。

说话人1: 问题六十一，为什么数据类型校验与业务规则校验不是一回事？例如，故障等级在格式上合法，但处理建议与故障类型不匹配，应该由哪个环节发现?
说话人1: 答，完全两个层面，校验的东西不一样。
说话人1: 数据类型校验，管的是“长什么样”，比如字段是不是数字、有没有值，是格式层面的。
说话人1: 业务规则校验，管的是“对不对”，比如故障等级是1级的时候，建议不能是“换设备”这么简单，要符合业务规则，是业务层面的。
说话人1: 格式合法不代表业务正确。这种不匹配，应该由业务规则校验环节发现，也就是Bean Validation之后的业务逻辑校验。
说话人1: 格式校验守门，业务校验兜底，各管一段。

说话人1: 问题六十二，如果希望模型返回一个枚举字段，如何保证返回值不仅是合法字符串，而且属于业务允许的取值范围?
说话人1: 答，两层校验，再加Prompt引导，三重保障。
说话人1: 第一层，Prompt里明确说明，该字段只能是哪几个值，比如“故障等级只能是LOW、MEDIUM、HIGH三者之一”，先引导模型输出正确值。
说话人1: 第二层，JSON Schema或者Java枚举类型约束，格式层面限制。
说话人1: 第三层，业务枚举校验，用Bean Validation的@Pattern或者自定义枚举校验器，不在枚举内就校验失败，触发重试。
说话人1: 三层下来，基本能保证值在允许范围内。

说话人1: 问题六十三，当模型可能生成不存在的设备编号、虚构的故障代码或不在知识库中的诊断结果时，仅靠结构化输出为什么无法解决事实性问题?
说话人1: 答，因为结构化输出只管格式对不对，不管内容真不真、对不对。
说话人1: 格式约束只能保证字段齐全、类型正确，保证不了字段里的内容是真实存在的、是正确的。不存在的设备编号，只要格式是字符串，结构化校验就会通过。
说话人1: 事实性问题，要靠外部知识库、靠工具调用、靠业务校验来解决，格式约束管不了这个。
说话人1: 不要以为结构化了就万事大吉，内容准确性是另外一个问题。

说话人1: 问题六十四，如何将结构化输出转换、Bean Validation校验、业务规则验证、数据库写入串联成一个可维护的服务流程?
说话人1: 答，分五步，层层递进，每层职责清晰。
说话人1: 第一步，结构化转换：调用模型，用Spring AI转换器转成目标Java对象，转换失败抛解析异常。
说话人1: 第二步，格式校验：Bean Validation校验字段、类型、必填，不合格抛校验异常。
说话人1: 第三步，业务规则校验：检查内容是否符合业务规则，比如设备是否存在、故障代码是否合法，不合格抛业务异常。
说话人1: 第四步，业务处理：执行对应的业务逻辑，比如创工单、派单。
说话人1: 第五步，数据落库：结果写入数据库。
说话人1: 每层只做自己的事，异常分层抛出，逻辑清晰，可维护性强。

七、异常处理、重试与可靠性

说话人1: 问题六十五，当模型返回非法 JSON、响应为空、字段缺失或类型错误时，应该如何划分解析异常、校验异常和业务异常？
说话人1: 答，按错误发生的阶段和原因划分，各归各类。
说话人1: 解析异常：发生在格式转换阶段，JSON语法错、没法解析、结构完全不对，属于模型输出格式问题。
说话人1: 校验异常：发生在Bean Validation阶段，格式合法，但缺字段、类型错、值不在范围，属于不符合契约。
说话人1: 业务异常：发生在业务规则校验阶段，格式校验都过了，但内容不符合业务规则，比如设备不存在。
说话人1: 分层抛异常，分层处理，排查的时候一看异常类型就知道哪段出问题了。

说话人1: 问题六十六，在什么情况下应该尝试修复模型输出，在什么情况下应该重新调用模型，在什么情况下应该直接失败并返回明确错误？
说话人1: 答，看错误类型和修复成本，分级处理。
说话人1: 尝试修复：小问题，比如外面包了Markdown代码块、多了点前缀后缀，清洗一下就能好，不用重调模型，成本低。
说话人1: 重新调用模型：格式问题大、缺字段、类型错，修复不了，就把错误告诉模型，让它重生成，重试1-2次。
说话人1: 直接失败：重试好几次都不行，或者业务错误，那就直接失败，返回明确错误码，不要无限重试。
说话人1: 原则：能修复就修复，不能修复就重试，重试不行就失败，成本和效果平衡。

说话人1: 问题六十七，如果解析失败后自动重试，如何避免无限重试、Token 浪费、延迟增长和费用失控？
说话人1: 答，加三个限制，把重试关进笼子里。
说话人1: 第一，最大重试次数，比如最多2次，不行就失败，不能无限循环。
说话人1: 第二，重试退避，每次重试间隔时间递增，避免瞬间集中重试。
说话人1: 第三，错误类型判断，只有格式相关的错误才重试，业务校验失败不重试，重试也没用。
说话人1: 还要记录每次重试的日志，监控重试率，太高了就优化Prompt或者换模型。
说话人1: 不能为了成功率，无限制重试，成本会爆炸的。

说话人1: 问题六十八，重试时应该将原始问题重新发送，还是将具体格式错误和校验失败原因传递给模型？两种策略各自有什么适用条件和风险？
说话人1: 答，两种策略，适用场景不一样。
说话人1: 重发原始问题：等于重新生成一次，优点是简单，缺点是大概率还会犯同样的错，适合随机错误，不适合系统性格式错误。
说话人1: 把错误原因告诉模型：相当于让模型改错，比如“字段faultCode缺失，请重新输出完整JSON”，模型会针对性修正，大概率下次就对了，适合系统性格式偏差。
说话人1: 风险：如果错误原因描述不好，模型可能理解偏了，反而更错。
说话人1: 一般第一次失败，带错误原因重试一次，效果最好。

说话人1: 问题六十九，为什么对模型输出执行字符串替换、删除 Markdown 标记或自动补括号等修复操作可能掩盖问题，甚至改变原始数据语义？
说话人1: 答，因为这些修复操作都是在猜测模型的意思，不一定猜对。
说话人1: 比如删Markdown，万一内容里真的有代码块内容，你删了就改了原意；
说话人1: 比如自动补括号，万一模型就是没写完，你补上了，内容就不对了；
说话人1: 比如字符串替换，替换错了，就把内容改了。
说话人1: 这些修复都是“急救措施”，偶尔用可以，但不能当常规操作，更不能掩盖模型输出本身的问题。不然哪天修复逻辑改了内容，出了业务问题，都不知道为什么。

说话人1: 问题七十，如果第一次调用返回合法结构但业务校验失败，第二次调用可能返回完全不同的诊断结果，应该如何设计结果一致性、审计与重试策略？
说话人1: 答，业务校验失败，不建议自动重试，因为模型每次输出可能不一样，重试也没用。
说话人1: 第一，业务校验失败的，直接返回校验失败，不要自动重试，因为内容问题重试大概率还是

错。
说话人1: 第二，如果要重试，必须把业务校验的错误原因也发给模型，明确告诉它哪里不对，让它修正，不是让它重新生成。
说话人1: 第三，审计：每次调用的输入输出、校验结果、重试次数，都要留日志，可审计。
说话人1: 第四，一致性要求高的场景，就不要靠模型，或者用工具调用查真实数据。
说话人1: 核心：内容错误，重试没用，要明确纠错；格式错误，才适合重试。

说话人1: 问题七十一，当模型响应被截断，如何区分输出 Token 上限造成的截断、网络传输异常和模型生成了不完整结构?
说话人1: 答，看三个特征，基本能区分。
说话人1: 第一，Token上限截断：一般内容到一半突然停了，结尾不完整，没有结束标记，通常是达到了模型的上下文或者输出Token限制，截断点很干脆。
说话人1: 第二，网络异常：一般是连接中断，抛IO异常，或者返回内容不完整，有乱码，伴随网络错误。
说话人1: 第三，模型生成不完整：比如JSON没闭合，说到一半停了，是模型自己生成中断，一般没有网络错误，重试可能又不一样。
说话人1: 可以结合错误日志、响应长度、异常类型一起判断。

说话人1: 问题七十二，如果业务要求高可用，应该如何为结构化输出链路设计超时、重试次数上限、降级策略、错误记录和监控指标?
说话人1: 答，全方位设计，保障可用性。
说话人1: 第一，超时：模型调用设置超时时间，比如30秒，避免一直等。
说话人1: 第二，重试次数：格式类错误最多重试2次，业务错误不重试。
说话人1: 第三，降级策略：结构化输出失败的话，降级成自然语言返回，或者返回默认错误结果，不要直接崩。
说话人1: 第四，错误记录：所有异常都记录日志，带traceId，方便排查。
说话人1: 第五，监控指标：解析成功率、校验通过率、重试率、平均耗时、错误分布，设阈值告警。
说话人1: 保证核心流程可用，出问题有兜底，有监控。

说话人1: 问题七十三，对于重要业务操作，为什么不能在模型输出一经解析成功后便直接执行？哪些操作仍需要权限检查、业务校验或人工审批?
说话人1: 答，因为结构化正确只代表格式对，不代表内容对、业务合法、安全。
说话人1: 还要做的检查：第一，权限检查，操作人有没有权限做这个事，不能模型说删就删。
说话人1: 第二，业务校验，操作符不符合业务规则，比如删除设备是不是符合流程。
说话人1: 第三，高风险操作必须人工审批，比如批量删除、设备控制，不能模型一句话就执行了。
说话人1: 格式校验只是第一步，业务安全才是底线。越重要的操作，越要多层校验，不能把模型输出当圣旨。

八、输出格式约束、安全与可信度

说话人1: 问题七十四，如果模型输出的 JSON 格式完全合法，但包含恶意 SQL、越权操作参数或不可信的指令内容，为什么依然不能直接执行?
说话人1: 答，因为格式正确和内容安全是两码事。格式校验只管结构对不对，不管内容有没有恶意。
说话人1: 比如模型返回的action是delete_device，参数是所有设备，格式完全正确，但执行了就是事故。
说话人1: 结构化输出只能保证你能解析成对象，不能保证对象里的内容是安全的、是符合权限的。
说话人1: 所以执行任何操作之前，必须做权限校验、业务校验、安全检查，不能因为格式对就放行。格式是载体，安全是底线。

说话人1: 问题七十五，在 AI 应用中，结构化输出与参数校验、身份认证、授权检查、SQL 参数化和防止命令注入之间是什么关系?
说话人1: 答，结构化输出是数据格式的基础，后面这些都是安全防护机制，分层保障。
说话人1: 结构化输出让你能把模型输出解析成对象，方便后续做校验；

说话人1: 身份认证确认是谁在操作；授权检查确认有没有权限做；
说话人1: 参数校验检查参数合法不合法；SQL参数化防止注入攻击。
说话人1: 结构化输出是第一层，是前提，后面的安全层一个都不能少。不能因为有了结构化输出，就跳过安全检查，那是本末倒置。

说话人1: 问题七十六，如果模型返回一个 action 字段，值为 delete_device，应用程序是否应该直接执行删除操作？如何设计允许操作列表和服务端权限检查？
说话人1: 答，绝对不能直接执行，模型说执行就执行那还得了。
说话人1: 设计原则：第一，白名单机制，只允许模型调用指定的、安全的操作，不在白名单的直接拒绝。
说话人1: 第二，权限校验，执行操作前，检查当前用户有没有对应权限。
说话人1: 第三，重要操作二次确认，或者人工审批。
说话人1: 第四，操作审计，所有操作都留日志，谁触发的、模型返回什么、执行结果是什么，可追溯。
说话人1: 模型只是建议，执行权在应用，不能反过来让模型指挥系统。

说话人1: 问题七十七，如果模型输出一个 URL、文件路径、设备编号或数据库查询条件，应如何防止合法格式中的恶意值绕过安全限制？
说话人1: 答，两层校验，格式校验过了之后，必须做业务安全校验。
说话人1: 第一，格式校验：先确保结构正确，字段类型对。
说话人1: 第二，合法性校验：每个字段都有合法范围，比如设备编号必须符合命名规则，URL必须是内部域名，路径不能跳出指定目录，查询条件不能有注入特征。
说话人1: 第三，权限校验：对应的数据、操作，当前用户有没有权限。
说话人1: 第四，输出转义：如果要拼SQL、拼命令，必须参数化，不能直接拼接。
说话人1: 格式是入门，安全是核心，每个字段都要做合法性校验，不能因为是结构化的就信任。

说话人1: 问题七十八，如果模型输出置信度、风险评分或概率，如何判断这些字段是否有可靠含义？为什么不能仅因为字段存在且类型正确，就将其视为经过校准的可信概率？
说话人1: 答，这些值都是模型自己估的，不是经过校准的真实概率，看看就行，不能当真。
说话人1: 为什么不可靠：第一，模型的置信度是自己拍脑袋的，没有统一标准，有的模型永远90%以上；第二，不同模型的分数没法比，没有统一刻度；第三，模型可能很自信但完全错了，也就是过度自信。
说话人1: 所以可以做参考，比如置信度低于多少就走人工审核，但不能把它当经过校准的概率，更不能只因为字段是数字就觉得很科学。
说话人1: 要有数：这是模型的自我感觉，不是客观测量值。

说话人1: 问题七十九，在涉及生产设备控制、医疗辅助、财务审批或其他高风险业务时，结构化输出约束能够降低哪些风险，又不能替代哪些安全控制？
说话人1: 答，能降低的风险：第一，解析错误风险，格式统一，不会因为解析错导致业务错误；第二，字段缺失风险，关键信息不会漏；第三，歧义风险，不会因为理解偏差导致操作错误。
说话人1: 不能替代的安全控制：第一，权限控制，谁能操作什么；第二，人工审批，高风险必须人过目；第三，业务规则校验，符不符合流程；第四，操作审计，所有操作留痕；第五，故障熔断，异常情况终止操作。
说话人1: 结构化输出是提升可靠性的工具，不是安全的替代品。高风险业务，该有的安全机制一个都不能少。

说话人1: 问题八十，如何确保模型返回的引用来源、证据列表或知识库文档 ID 确实存在，并且与诊断结论之间具有可验证的关联？
说话人1: 答，两步走，检索校验+关联验证。
说话人1: 第一步，检索校验：所有引用的文档ID、来源，都去真实知识库里面查一遍，是不是真的存在，不存在的就标记无效，或者让模型重新生成。
说话人1: 第二步，关联验证：检查结论是不是能从引用的证据里推导出来，有没有断章取义，有没有无中生有。
说话人1: 不能模型说引用了什么就信什么，必须自己核实。尤其是RAG场景，引用校验是必须有的一步。不然模型瞎编一个来源，你也直接用吗？

九、测试、验收与可观测性

说话人1: 问题八十一，如何为结构化输出功能设计单元测试，以验证正常 JSON、字段缺失、类型错误、非法枚举、未知字段和空响应等情况？
说话人1: 答，针对每种异常场景构造测试用例，覆盖边界。
说话人1: 第一，正常用例：标准正确输出，验证能正常解析，校验通过。
说话人1: 第二，缺字段用例：缺少必填字段，验证抛出校验异常。
说话人1: 第三，类型错误用例：数字写成字符串，验证解析或者校验失败。
说话人1: 第四，非法枚举用例：枚举值不在范围内，验证校验失败。
说话人1: 第五，未知字段用例：多了额外字段，验证能正常忽略或者告警。
说话人1: 第六，空响应、非法JSON用例：验证抛出解析异常。
说话人1: 每种场景都有预期结果，全部通过才算合格。

说话人1: 问题八十二，如何编写集成测试，验证 Spring AI 从请求构造到模型响应解析的完整链路，而不是只验证模拟数据能够转换为 Java 对象？
说话人1: 答，端到端集成测试，走完整链路，不用Mock模型，或者用官方测试模型。
说话人1: 第一步，配置真实的ChatClient和测试模型，比如用小模型或者测试专用模型。
说话人1: 第二步，构造不同类型的请求，正常的、模糊的、极端的，都调用真实的模型接口。
说话人1: 第三步，验证返回的对象是不是正确、校验是不是通过、异常是不是符合预期。
说话人1: 第四步，验证异常场景，比如故意提不可能的要求，看错误处理对不对。
说话人1: 这样才能验证完整链路是不是通的，而不是自己Mock出来的假成功。

说话人1: 问题八十三，如果测试环境使用 Mock 模型，而生产环境使用真实模型，哪些格式问题可能无法通过 Mock 测试发现？
说话人1: 答，很多真实模型的问题Mock都测不出来，因为Mock是你控制输出的。
说话人1: 第一，模型输出的格式随机性，真实模型有时候对有时候不对，Mock永远正确，测不出容错能力。
说话人1: 第二，Markdown包裹、多余解释这些真实模型常见的问题，Mock不会模拟。
说话人1: 第三，不同模型的服从度差异，Mock都听话，真实模型可能不听话。
说话人1: 第四，长文本、复杂场景下的格式漂移，Mock也模拟不出来。
说话人1: 所以Mock只能测逻辑对不对，真实效果必须用真实模型跑集成测试。

说话人1: 问题八十四，如何设计一组结构化输出测试样例，覆盖正常输入、模糊输入、极端输入、长输出、恶意输入和模型异常响应？
说话人1: 答，六类测试用例，全覆盖。
说话人1: 正常输入：标准问题，预期输出格式正确。
说话人1: 模糊输入：表述不清晰的问题，看模型能不能按要求结构输出。
说话人1: 极端输入：很长的问题、很多字段要求，看会不会超上下文、会不会格式崩。
说话人1: 长输出：需要返回很多内容、长文本的场景，看会不会截断。
说话人1: 恶意输入：要求输出恶意SQL、越权操作，看会不会被引导，输出是不是符合安全预期。
说话人1: 模型异常：模拟模型返回非法格式、缺字段，看程序的异常处理对不对。
说话人1: 覆盖了这几类，基本就能应对线上的大部分情况。

说话人1: 问题八十五，如何衡量输出格式约束的有效性？应该区分格式合法率、Schema 符合率、字段完整率、业务校验通过率和事实准确率吗？
说话人1: 答，必须分维度衡量，不能笼统说“好用”。
说话人1: 格式合法率：输出是合法JSON的比例，衡量基础格式能力。
说话人1: Schema符合率：字段、类型、必填都符合Schema的比例，衡量结构符合度。
说话人1: 字段完整率：必填字段缺失的比例，衡量信息完整性。
说话人1: 业务校验通过率：业务规则校验通过的比例，衡量内容合规性。
说话人1: 事实准确率：内容事实正确的比例，衡量输出正确性。
说话人1: 五个维度，从格式到内容，逐层深入，才能真实反映效果。不能只看成功率，要看哪层成功率。

说话人1: 问题八十六，当生产环境出现大量解析错误时，应该记录哪些诊断信息？如何在便于排查与保护用户隐私之间做出权衡？
说话人1: 答，记录必要的元数据和异常信息，不记录敏感内容。
说话人1: 要记录的：第一，异常类型，解析错还是校验错；第二，错误摘要，比如哪个字段错了、什么类型错；第三，traceId，能关联到完整请求；第四，模型版本、模型供应商；第五，请求的大致类型，不记录具体内容。
说话人1: 不要记录的：用户具体输入输出内容、敏感信息、个人数据。
说话人1: 原则：排查问题需要的元数据要记，具体业务内容不要记，既方便排查，又保护隐私。

说话人1: 问题八十七，如果模型供应商或者版本发生变化，如何验证原有结构化输出契约依然成立？是否需要建立回归测试集？
说话人1: 答，必须做回归验证，必须有回归测试集。
说话人1: 第一，准备一套固定的回归测试用例，覆盖各种典型场景。
说话人1: 第二，换模型之后，用同样的测试用例跑一遍，看各个指标有没有下降。
说话人1: 第三，格式合法率、Schema符合率、业务校验通过率，和之前的基线对比，下降超过阈值就不行。
说话人1: 不能想当然认为换个模型还能用，必须用测试集验证。接口契约是严肃的，不能说换就换。

说话人1: 问题八十八，如何通过日志、指标与链路追踪，定位结构化输出失败究竟发生在模型生成、响应转换、数据校验还是业务执行环节？
说话人1: 答，全链路埋点，每个环节都打日志、挂指标，出问题看一眼就知道在哪。
说话人1: 第一，模型生成环节：记录模型原始返回、格式是否合法，出问题就是模型侧。
说话人1: 第二，响应转换环节：记录转换是否成功、解析异常，出问题就是转换器、类型定义的问题。
说话人1: 第三，数据校验环节：记录校验通过率、错误字段，出问题就是校验不通过。
说话人1: 第四，业务执行环节：记录业务异常，出问题就是业务规则的事。
说话人1: 再加上traceId串起来，从头到尾哪个环节错了，一目了然。

十、架构设计与综合面试题

说话人1: 问题八十九，假设你要开发一个企业制造设备故障问答助手，模型需要同时返回面向用户的自然语言解释和供系统使用的结构化故障诊断结果，你会如何设计输出契约？
说话人1: 答，双输出设计，分层返回，各有用途。
说话人1: 第一，统一的响应对象，包含两个部分：userAnswer字段，字符串，给用户看的自然语言解释；diagnosis字段，结构化的FaultDiagnosis对象，给系统用的，包含故障类别、等级、原因、建议。
说话人1: 第二，Prompt里明确要求模型同时输出自然语言回答和结构化诊断结果，两部分对应。
说话人1: 第三，返回给前端的时候，自然语言部分展示给用户，结构化部分系统用来做后续流程，比如创工单、派单。
说话人1: 这样既满足用户体验，又满足系统自动化需求，两不误。

说话人1: 问题九十，如果模型既要回答用户的问题，又要判断是否需要创建维修工单，应该采用一个包含 answer 和 shouldCreateWorkOrder 等字段的统一响应对象，还是把回答与动作决策拆开？如何权衡？
说话人1: 答，推荐拆开，分阶段，不要混在一个对象里，职责清晰。
说话人1: 第一，第一步先做诊断，返回诊断结果和回答，这是第一层。
说话人1: 第二，第二步单独做决策判断，根据诊断结果、故障等级、设备情况，判断要不要创建工单。
说话人1: 为什么不混在一起？因为决策逻辑是业务规则，不应该让模型完全决定，模型只负责诊断，创不创工单是业务规则决定的。
说话人1: 这样权责清晰，模型管诊断，业务管决策，模型换了，决策逻辑也不用改，解耦。不要什么都让模型干，业务系统的归系统。

说话人1: 问题九十一，如果结构化响应需要同时支持多个模型供应商，如何设计统一的 Java 输出契约与供应商适配层，避免业务逻辑依赖某个供应商的专有参数?
说话人1: 答，核心是面向接口编程，加适配层，屏蔽供应商差异。
说话人1: 第一，定义统一的Java输出对象，也就是业务契约，比如FaultDiagnosis，业务代码只依赖这个对象，不依赖具体厂商。
说话人1: 第二，每个供应商做一个适配层，负责把该厂商的输出，转换成统一的Java对象。
说话人1: 第三，统一的校验层，不管哪个厂商来的，都过统一的业务校验。
说话人1: 第四，配置化切换，用哪个供应商靠配置，不用改业务代码。
说话人1: 这样换供应商，只加个适配层就行，业务逻辑不动，符合开闭原则。

说话人1: 问题九十二，如果一个复杂工作流中，模型 A 的输出是模型 B 的输入，而模型 B 的输出又决定是否调用工具，应该如何设计各阶段的 Schema、验证、错误处理和状态流转?
说话人1: 答，流水线式设计，每个阶段都是输入输出契约+校验+异常处理。
说话人1: 第一，每个阶段定义独立的Schema和Java对象，作为阶段契约。
说话人1: 第二，每个阶段输出之后，都做格式校验+业务校验，不合格就走异常处理，不流入下一级。
说话人1: 第三，错误处理：每个阶段的错误分等级，可重试的重试，不可重试的终止流程，返回错误。
说话人1: 第四，状态流转：用状态机管理，每个阶段的结果对应下一个阶段的输入，清晰不混乱。
说话人1: 相当于每个节点都有契约，层层把关，层层推进。

说话人1: 问题九十三，当结构化输出字段经常增加、修改或废弃时，如何管理 Schema 版本、向后兼容、默认值和旧数据处理?
说话人1: 答，版本化管理，兼容优先。
说话人1: 第一，Schema版本号，每次变更升级版本，比如v1、v2，不要直接改旧的。
说话人1: 第二，向后兼容：新增字段必须有默认值，不能必填，不然旧版本的输出就解析失败了。
说话人1: 第三，废弃字段：先标记废弃，保留兼容，几个版本之后再移除，不要直接删。
说话人1: 第四，旧数据处理：历史数据做兼容转换，或者保留对应版本的解析器。
说话人1: 和API版本管理一样，不能说改就改，要考虑向下兼容。

说话人1: 问题九十四，如果一个业务流程既要求模型生成严格结构化结果，又需要保留模型的原始回答用于审计，如何设计原始响应、解析对象和最终业务结果之间的关系?
说话人1: 答，三层存储，各有用途，全部留痕。
说话人1: 第一层，原始响应：模型返回的原始文本，完整保存，用于审计、排查问题，原封不动。
说话人1: 第二层，解析对象：结构化解析后的Java对象，用于业务处理。
说话人1: 第三层，最终业务结果：经过业务校验处理之后的最终结果。
说话人1: 三者用同一个traceId关联，都存日志或者数据库，可追溯，可审计。
说话人1: 出问题了可以回去看原始输出，是模型错了，还是解析错了，还是业务处理错了，一目了然。

说话人1: 问题九十五，假设你正在代码审查一个 Spring AI 项目，发现开发者通过 content() 获取模型文本后直接调用 objectMapper.readValue()，并把结果写入数据库。你会检查哪些潜在问题?
说话人1: 答，这是非常不规范的写法，问题很多，至少查七个点。
说话人1: 第一，没有格式约束，模型输出格式不稳定，随时可能解析失败。
说话人1: 第二，没有异常处理，解析失败直接崩。
说话人1: 第三，没有校验，字段对不对、值合不合格，直接写库，数据质量没保障。
说话人1: 第四，没有结构化输出转换器，自己手动解析，重复造轮子，不规范。
说话人1: 第五，没有业务校验，内容对不对直接写，脏数据入库。
说话人1: 第六，异常、重试、降级机制都没有，鲁棒性差。
说话人1: 第七，原始响应不保留，出问题没法排查。
说话人1: 这是典型的“能跑就行”的写法，生产级绝对不能这么干。

说话人1: 问题九十六，面试官要求你解释：使用 Prompt 要求 JSON、使用 Spring AI 输出转换器、使用供应商原生结构化输出、执行 Java Bean Validation，这四种措施分别解决什么问题，又为什么通常需要组合使用?
说话人1: 答，四个措施在不同层面，层层加固，组合起来才是完整的解决方案。
说话人1: Prompt要求JSON：是软引导，让模型知道要输出JSON，最基础，成本最低，效果有限。

说话人1: 供应商原生结构化输出：是模型服务端硬约束，强制输出合法JSON，大幅提升格式正确率，是基础保障。
说话人1: Spring AI输出转换器：是框架适配层，把模型输出转成Java对象，统一转换逻辑，简化开发。
说话人1: Java Bean Validation：是业务校验层，检查字段、值、规则符合不符合业务要求，把关内容质量。
说话人1: 四层分别是：引导、强制格式、转换、业务校验，一层管一段，组合起来，从格式到内容，从生成到转换，全方位保障。

说话人1: 问题九十七，如果要求你从零实现一条生产级结构化输出链路，你会如何划分模型调用层、输出转换层、格式校验层、业务验证层、异常恢复层与可观测性层?
说话人1: 答，六层架构，层层负责，边界清晰。
说话人1: 第一层，模型调用层：对接大模型API，负责调用模型、处理模型异常、重试。
说话人1: 第二层，输出转换层：负责把模型输出转换成Java对象，格式清洗、解析、类型映射。
说话人1: 第三层，格式校验层：Schema校验、Bean Validation，检查格式、字段、类型、必填。
说话人1: 第四层，业务验证层：业务规则校验、权限校验、安全检查。
说话人1: 第五层，异常恢复层：重试、降级、容错、错误处理。
说话人1: 第六层，可观测性层：日志、指标、链路追踪、告警。
说话人1: 每层只做自己的事，职责清晰，可替换可扩展。

说话人1: 问题九十八，如果模型经常返回格式正确但内容不完整的对象，你会如何区分 Prompt 约束不足、Schema 定义不合理、模型能力限制和业务需求表达不清，并通过证据确定改进方向?
说话人1: 答，逐个排查，用证据说话，不要拍脑袋。
说话人1: 第一步，先看Prompt：有没有明确说清楚字段、类型、要求，是不是描述模糊。优化Prompt再跑，看有没有改善，有改善就是Prompt的问题。
说话人1: 第二步，看Schema定义：有没有漏字段、类型定义不对、约束不清晰。调整Schema再试，有改善就是Schema的问题。
说话人1: 第三步，前两个都没问题，换个更强的模型试试，好了就是模型能力不够。
说话人1: 第四步，都没问题还不行，那就是业务需求没说清楚，内部对齐需求。
说话人1: 用控制变量法，每次改一个变量，看效果，就能定位根因。

说话人1: 问题九十九，如果业务负责人要求“只要模型返回 JSON，就保证系统绝不出错”，你会如何从接口契约、类型校验、业务规则、安全控制和分布式系统可靠性等角度分析这一要求?
说话人1: 答，这是典型的对AI系统认知不足，把格式正确等同于系统正确。
说话人1: 第一，接口契约层面：JSON只是格式，契约不止格式，还有字段、类型、枚举、业务规则，JSON只是基础。
说话人1: 第二，类型校验层面：JSON格式对，不代表类型对、字段全，还要Bean Validation。
说话人1: 第三，业务规则层面：就算字段都对，内容也可能错，还要业务校验。
说话人1: 第四，安全控制层面：格式正确不代表内容安全，越权、注入、恶意指令都可能有，还要安全校验。
说话人1: 第五，分布式可靠性：模型本身就有概率性，还要重试、降级、熔断，保证系统可靠性。
说话人1: 所以“返回JSON”只是第一步，离“系统绝不出错”差远了。完整的系统是多层防护的，不是靠一个格式就万事大吉。

说话人1: 问题一百，请用一个完整的 Java AI 项目示例，说明为什么要约束大模型输出格式，并从自由文本输出的缺点讲起，逐步解释结构化数据契约、Spring AI 输出转换、Java 类型映射、数据校验、异常处理与生产级验收之间的关系。
说话人1: 答，就用设备故障诊断助手的例子，一步步演进，讲清楚为什么要做这些。
说话人1: 最开始，模型直接返回自然语言回答，用户能看懂，但程序没法直接用，要提取故障信息，只能写正则，经常错，维护难——这就是自由文本的缺点，程序处理不了。
说话人1: 然后，我们要求模型输出JSON格式，程序能解析了，但模型有时候输出不标准，缺字段、类型错，解析经常失败——这就是只有软约束的问题，不稳定。
说话人1: 然后，我们用模型原生JSON Mode，加上Spring AI的结构化输出转换器，直接映射成Java对象，解析成功率大幅提升，开发也简单了——这就是框架和原生结构化能力的价值。
说话人1: 接着，我们加上Bean Validation，校验必填、类型、枚举范围，保证数据符合契约——这就是

校验层的价值，格式对还要规则对。
说话人1: 然后，加上业务规则校验，检查设备是否存在、故障代码是否合法，保证内容业务正确——这就是业务校验，格式对不代表内容对。
说话人1: 再然后，加上异常处理、重试、降级，保证系统稳定性——这是可靠性保障。
说话人1: 最后，加上日志、指标、监控、审计，实现可观测可审计——这就是生产级。
说话人1: 从自由文本，到结构化，到校验，到业务，到可靠，到可观测，一步步升级，最终形成生产级的AI应用。不是一步到位的，也不是只要一个JSON就解决的。

说话人1: 全部回答完毕。